



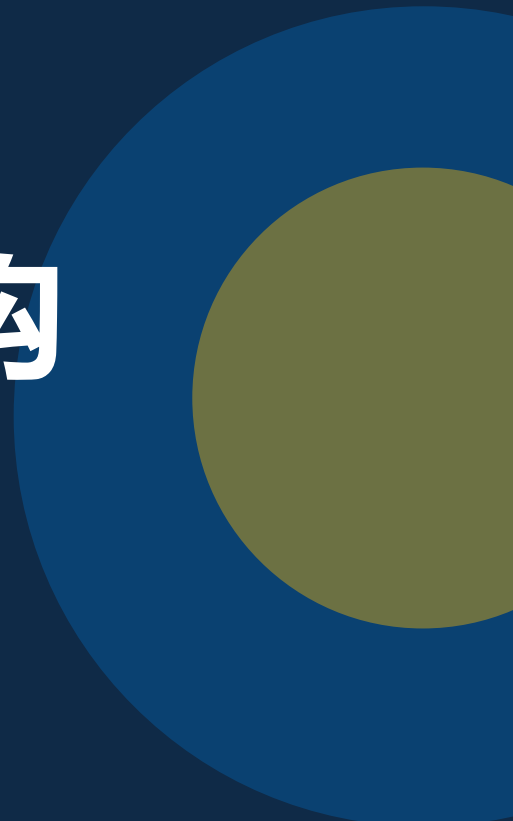
A G E N T H A R N E S S

构建下一代 AI 智能体的核心架构

从 Azure Foundry、Anthropic 到 AWS Bedrock 的工程实践与设计哲学

Kinfey Lo – Microsoft Senior Cloud Advocate

2026



本次内容议程

本次将分六大主题深入探讨 Agent Harness 的设计与实践

01	概念基础	什么是 Agent Harness 及其核心组件
02	三大平台	Azure Foundry、Anthropic、AWS Bedrock 详解
03	架构对比	隔离模型、协议、身份与安全多维度比较
04	实战指南	开发流程、上下文工程与性能优化
05	趋势展望	Meta-Harness 与可演进架构的未来
06	Q&A	深入交流与现场答疑

PART 01

概念基础

理解 Agent Harness: 从 Prompt 到工程化的关键跃迁



为什么我们需要 Agent Harness

构建生产级 AI 智能体涉及大量横切关注点,远不止 Prompt 工程

容器化部署

镜像构建、Web 服务器配置、运行时编排

安全治理

身份管理、最小权限、敏感数据保护

状态持久化

会话内存、文件存储、跨轮次状态恢复

弹性伸缩

并发会话、空闲回收、冷启动时延

可观测性

日志聚合、指标采集、追踪与诊断

版本回滚

金丝雀发布、蓝绿部署、流量切换

Agent Harness 是一套将这些复杂性收敛到平台层的工程抽象

什么是 Agent Harness

Harness = 围绕 LLM 的执行环境、循环逻辑与基础设施集合

Agent Harness 三层抽象

Brain — 大脑

LLM 推理 + Harness 循环逻辑

Hands — 手

沙箱执行环境 + 工具调用接口

Session — 会话

追加式事件日志 + 状态恢复机制

核心定义

Agent Harness

是一套围绕 LLM 构建的工程框架,负责调用模型、路由工具调用、维护状态、保障安全。

类比操作系统

Harness 之于 LLM,如同 OS 之于 CPU——通过虚拟化与抽象,把硬件复杂度收敛到平台层。

演进规律

Harness 编码了对 LLM 能力的假设。模型变强后,部分假设会过时——因此接口需稳定,实现可演进。

来源:Anthropic Engineering — Scaling Managed Agents (2026)

Agent Harness 的三大核心组件

Anthropic 提出的虚拟化模型:Brain / Hands / Session 解耦设计

Brain — 大脑

LLM 推理与控制循环

- 调用 Claude / GPT / 自研模型完成推理
- 解析输出、路由工具调用
- Stateless 设计,可水平扩展
- 失败时通过 wake(sessionId) 重启

Hands — 手

沙箱与工具执行

- execute(name, input) → string 通用接口
- 容器、函数、MCP Server、外部 API 均可
- 可插拔——可换为 Phone、模拟器或外部系统
- 失败被 Brain 当成普通工具错误处理

Session — 会话

持久化事件日志

- append-only 事件流,记录所有交互
- getEvents() 允许 Brain 回看上下文
- 独立于 Brain 与容器存活
- 支持崩溃恢复与跨设备访问

解耦的力量:大脑、手、会话的分离

通过松耦合接口,让任一组件可独立失败、替换与扩展

× 耦合架构(早期方案)

- × Brain、Hands、Session 同处一个容器
- × 容器失败 → 整个会话丢失
- × 无法连接到客户的 VPC 资源
- × 调试困难,失败原因难定位
- × TTFT 受容器冷启动严重拖慢
- × 凭据泄漏可被 Prompt Injection 利用

✓ 解耦架构(Managed Agents)

- ✓ 三组件独立部署,各自可失败可重启
- ✓ Brain 无状态,水平扩展极简
- ✓ Hands 仅按需创建,不挡 TTFT
- ✓ 可连接客户 VPC,Brain 不需 peering
- ✓ 凭据不进沙箱,通过代理访问
- ✓ P50 TTFT 下降 60%,P95 下降 90%+

"Don't adopt a pet" — 把容器当牛而非宠物

AI Agent 的三阶段演进

从 Prompt 模板到托管运行时,工程化能力逐级递进



Stage 1

Prompt 模板

通过 System Prompt + Few-shot 完成单轮任务



Stage 2

自定义 Agent 框架

LangChain / Semantic Kernel / Agent Framework, 自管编排循环



Stage 3

托管 Agent 平台

Azure Foundry / Anthropic / AWS Bedrock 提供 Harness 即服务

当前焦点: Stage 3 —— 各大云厂商正激烈竞争 Managed Harness 标准

PART 02

三大平台详解

Microsoft Foundry · Anthropic Managed Agents · AWS Bedrock



三大托管 Agent 平台总览

代表三种不同的设计哲学和工程取舍

AZ Microsoft Foundry Hosted Agents (Preview) • BYOC:容器即代码,框架不限 • Responses + Invocations 双协议 • Per-session VM 隔离沙箱 • Microsoft Entra ID 自动绑定 • 深度集成 Foundry Toolbox	AN Anthropic Managed Managed Agents • Brain / Hands / Session 三层解耦 • Meta-Harness 思想,接口稳定 • 面向长程任务,会话独立持久化 • MCP 工具与代理凭据隔离 • 工程博文公开了详细设计	AWS AWS Bedrock Managed Agents (OpenAI) • 首批 OpenAI Harness 商用 • 依托 AgentCore 默认计算环境 • 数据不出 AWS,身份审计内建 • 贴近现有 AWS 数据与计算服务 • Limited Preview 阶段
---	--	--

Microsoft Foundry Hosted Agents

Microsoft 推出的托管智能体平台,支持自带框架与代码

平台定位

将 Agent 作为容器化应用,运行于 Microsoft 托管的基础设施。开发者保留代码与框架的完全控制,平台负责伸缩、状态、可观测性与生命周期。

适用场景

- ▶ 自带代码 / 框架(Agent Framework、LangGraph、Semantic Kernel)
- ▶ 需自定义协议接收 Webhook 或非 OpenAI 负载
- ▶ 需控制计算资源(CPU/Memory)
- ▶ 需有状态工作负载(跨轮持久化)

BYOC

自带容器镜像

Auto-Scale

弹性伸缩 + 空闲零成本

VM Isolation

每会话独立 VM 沙箱

Entra ID

Agent 专属身份与端点

Azure Foundry 工作机制

从镜像构建到运行时调用的完整生命周期



平台自动管理:

- 容器生命周期:活动时按需配置,空闲超时(15 分钟)自动卸载
- 状态持久化:\$HOME 与 /files 内容在会话恢复时自动还原
- 弹性伸缩:支持 0.25 vCPU / 0.5 GiB 至 2 vCPU / 4 GiB 沙箱
- 可观测性:OpenTelemetry 集成,日志、追踪与指标内建

Responses 与 Invocations 双协议

两种协议覆盖对话、Webhook、批处理与定制场景

Responses 协议

适用	对话式机器人、多轮 Q&A、RAG、后台异步
格式	OpenAI 兼容的 /responses 协议
客户端	任何 OpenAI SDK(Python / JS / C#)即开即用
会话	平台托管对话历史(conversation ID)
流式	ResponseEventStream,内建生命周期事件
后台	background: true + 平台轮询取消

Invocations 协议

适用	Webhook 接收、分类抽取、自定义流式协议
格式	任意 JSON 入,任意 JSON 出
客户端	自定义 SDK,需自行约定契约
会话	应用层管理(内存 / Cosmos DB)
流式	原生 SSE,完全自控
后台	需自行追踪任务并实现轮询端点

两种协议可同时启用,亦可叠加 Activity(Teams/M365)与 A2A(Agent-to-Agent)协议

Agent 身份与端点

每个托管 Agent 自动获得专属 Entra ID 与端点

两种身份的角色分工

Microsoft Entra ID(Agent 身份)

- 部署时自动创建,每个 Agent 独立
- 容器运行时鉴权,调用模型与下游服务
- 通过 azd 自动赋予 Azure AI User 角色

Project Managed Identity(项目级)

- Foundry 项目系统分配身份
- 平台基础设施操作专用
- 例:Container Registry Reader
- 与 Agent 运行时身份分离

自动暴露的端点

Responses Endpoint

```
{project_endpoint}/agents/{name}/  
endpoint/protocols/openai/v1/responses
```

Invocations Endpoint

```
{project_endpoint}/agents/{name}/  
endpoint/protocols/invocations
```

OBO(On-Behalf-Of):Teams/M365 集成时可代用户调用下游服务

会话(Sessions)与对话(Conversations)

两种状态抽象服务于不同协议

维度	Session(会话)	Conversation(对话)
定义	包含 \$HOME 与 /files 的逻辑会话	Foundry 中存储的对话历史记录
持久化	\$HOME 与 /files 跨轮持久化	消息、工具调用与响应
生命周期	30 天上限,15 分钟空闲卸载	独立于计算状态长期存储
主导协议	Invocations 协议主控	Responses 协议主控
跨渠道	受会话 ID 绑定	Playground / API / Teams 共享
管理 API	列出 / 终止 / 上传文件	Foundry 自动维护

会话计算生命周期:Active → 15 min idle → 状态持久化 → 同 ID 再请求时还原计算

安全与数据治理

每会话 VM 隔离 + 多重安全边界设计

Per-Session VM Isolation

- ✓ 每个会话独立 VM 沙箱
- ✓ 持久化 \$HOME 与 /files 文件系统
- ✓ Scale-to-zero 同时支持有状态恢复
- ✓ 会话间完全隔离

私有网络支持

Hosted Agents 可部署于网络隔离的 Foundry 资源中。注意：存放镜像的 Azure Container Registry 目前仍需开放公网端点，私网 ACR 暂不支持。

凭据与密钥管理

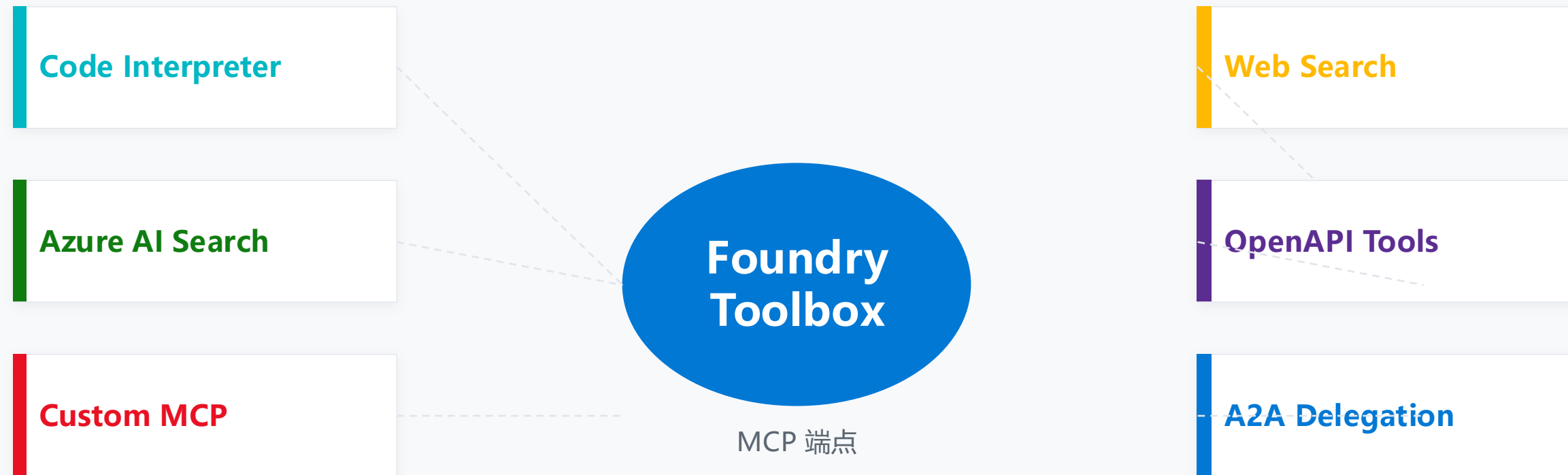
- ⚠ 切勿将密钥写入镜像或环境变量
- ⚠ 使用 Managed Identity 与 Connections
- ⚠ Secrets 集中存储于 Azure Key Vault
- ⚠ 私网部署需手工 RBAC 授权 Agent ID

第三方系统风险

接入非 Azure Direct Models 或第三方 MCP 时，数据可能流出 Azure 合规边界。需逐一审视数据共享、保留与位置政策。

Foundry Toolbox 与平台扩展

通过 MCP 端点接入丰富的内建工具集



Agent 代码通过标准 MCP 客户端连接 Toolbox 端点 — 平台不会自动注入工具,开发者拥有完全控制

Anthropic Managed Agents

Claude Platform 的托管服务,面向长程 Agent 任务而生

"Harnesses encode assumptions that go stale as models improve."

Harness 编码了对模型能力的假设,这些假设会随模型变强而过时。Managed Agents 围绕一组稳定接口构建,让接口跨越具体实现长存。

01

稳定接口

Stable Interfaces

类比 OS 的 read() — 上层接口不变,底层实现自由演进

02

组件解耦

Decoupled Components

Brain / Hands / Session 三层分离,可独立失败与替换

03

Meta-Harness

Meta-Harness Design

对接口有主张,对底层实现保持中立

三大接口的关键 API

通过最小接口集合让组件可独立演化

Brain	<code>wake(sessionId)</code> 唤醒新 Harness 进程,从会话恢复	<code>execute(name, input) → string</code> 调用任意 Hand,工具调用统一抽象	<code>emitEvent(id, event)</code> 向会话日志写入持久事件
Hands	<code>execute(name, input) → string</code> Brain 看不见底层实现细节	<code>provision({resources})</code> 按需配置容器/计算资源	失败 → <code>tool-call error</code> 容器死亡被 Brain 重试或路由
Session	<code>emitEvent(id, event)</code> append-only 事件追加	<code>getEvents()</code> 按位置切片回看上下文	<code>getSession(id)</code> 崩溃后重建 Brain 的执行上下文

Source: Anthropic Engineering Blog, "Scaling Managed Agents" (2026-04-08)

结构化的安全边界设计

凭据永不进入沙箱:Resource-Bundled Auth + Vault

× 风险:凭据与代码同处沙箱

Prompt Injection 只需说服 Claude 读取自己的环境,即可拿到凭据。一旦攻击者持有 Token,可衍生不受限的会话,并将工作委派给它们。

✓ 修复:凭据物理隔离

结构性修复保证 Token 永不可达沙箱。即便 Claude 越来越聪明,也无法从沙箱内访问到任何凭据。

Resource-Bundled Auth

凭据与资源捆绑

Git 例:在沙箱初始化阶段使用仓库 Token 完成 clone,并将 Token 注入本地 git remote。沙箱内的 push/pull 工作正常,Agent 不需直接处理 Token。

Vault + MCP Proxy

凭据存于沙箱外的金库

OAuth Token 存于安全 Vault。Claude 通过专用代理调用 MCP 工具,代理凭会话 Token 取出对应凭据,再调下游服务。Harness 全程不知凭据。

Many Brains, Many Hands 架构

解耦带来的扩展红利:延迟降低 + VPC 自由

Many Brains — 多大脑

Brain 与容器解耦后,大量 Brain 仅是无状态进程,启动多 Brain 即开多进程。容器仅按需通过 `execute()` 工具调用配置,不再为永远不接触沙箱的会话支付容器启动成本。

Many Hands — 多手

`execute(name, input) → string` 接口可承载任何工具:容器、MCP Server、自研工具。Hand 不绑定特定 Brain,因此 Brain 可以将 Hand 互相传递。

性能成果:Time-To-First-Token 显著优化

60%

P50 TTFT 下降

90%+

P95 TTFT 下降

0

无谓容器启动等待

Session ≠ Claude 的上下文窗口

可恢复的上下文存储 + 可定制的上下文工程

传统做法的瓶颈

- × 长程任务超出 Claude 上下文窗口
- × Compaction(压缩)做不可逆决策
- × Context trimming 会丢弃可能再用的 Token
- × 如果未存储,被压缩消息无法恢复
- × REPL 中保存上下文但绑定沙箱寿命

Managed Agents 的解法

- ✓ Session = 持久化事件流,生存期独立
- ✓ getEvents() 提供位置切片接口
- ✓ Brain 可回看任意历史片段
- ✓ Harness 可在写入 context 前自由变换
- ✓ 存储与上下文工程关注点彻底分离
- ✓ 兼顾 Prompt Cache 命中率优化

AWS Bedrock Managed Agents

搭载 OpenAI Harness 的 AWS 托管智能体平台

Bedrock Managed Agents, powered by OpenAI

将前沿 AI 模型与可信的 AWS 基础设施结合,使客户能在云中快速构建生产级、由 OpenAI 驱动的智能体。

前沿智能

采用 OpenAI Harness 释放前沿模型潜力,执行更快、推理更准、长任务可控。

默认安全

每个 Agent 拥有自己身份并审计每次行动。运行于客户环境,数据从不离开 AWS。

AWS 规模

可直连 AWS 计算与数据服务,贴近企业级数据与系统部署。

AgentCore 集成

Bedrock AgentCore 是默认计算环境,提供策略、发现、可观测性能力。

AWS Bedrock 关键能力详解

OpenAI Harness + AgentCore 双引擎驱动

OpenAI Harness 内核

推理速度

针对前沿模型深度优化执行路径

推理质量

释放模型的更锐利推理能力

长程任务

可靠的长时操控与状态保持

运行时

推理 + 内存 + 技能在客户环境内统一管理

AgentCore 默认运行环境

开放平台

支持任何模型与框架构建 Agent

授权策略

策略执行 + 工具权限统一管理

Agent 发现

Agent 与 Agent Tool 服务发现

可观测性

评估与运维监控能力随企业规模扩展

PART 03

架构对比

从隔离模型到协议、安全、观测全方位横评



三平台架构对比矩阵

维度 × 平台:一张表读懂关键差异

维度	Azure Foundry	Anthropic	AWS Bedrock
运行模式	BYOC 容器	托管解耦组件	Managed + AgentCore
框架支持	Agent FW / LangGraph / SK	Claude SDK	OpenAI Harness
隔离单元	Per-session VM 沙箱	Sandbox 容器(可换)	客户 AWS 环境
主协议	Responses + Invocations	Brain/Hand/Session API	OpenAI 兼容
身份模型	Microsoft Entra ID	MCP Token + Vault	Agent 独立身份
状态层	Session + Conversation	Session 事件流	运行时托管
可观测性	OpenTelemetry 内建	事件流可回溯	AgentCore 监控
阶段	Preview	公开发布	Limited Preview

容器化与隔离模型对比

三种隔离哲学对应三种工程权衡

Azure: VM 级会话隔离

Per-session VM Sandbox

- 每个会话独立 VM, 极强隔离边界
- \$HOME 与 /files 自动持久化
- Scale-to-zero + 状态恢复并存
- 冷启动可预测, 适合企业合规

Anthropic: 容器即牛

Container as Cattle

- Sandbox 是 execute() 工具的实现
- 失败被 Brain 当作工具错误处理
- 可在客户 VPC 中运行 Hand
- Brain 与容器解耦, 可灵活替换

AWS: AgentCore 默认环境

Customer Environment

- Agent 运行于客户 AWS 环境
- 数据从不离开 AWS
- 依托 AWS 计算/存储/网络
- AgentCore 提供横向扩展能力

协议与接口的工程哲学差异

OpenAI 兼容 vs. Meta-Harness 内部 API vs. 双协议混合

Azure: 双协议并存

优势

- ✓ Responses 即开即用
- ✓ Invocations 灵活定制
- ✓ 可叠加 Activity / A2A

局限

- ✗ 需选择主协议
- ✗ 并存增加运维复杂度

Anthropic: 内部 API 设计

优势

- ✓ Brain/Hand/Session 接口稳定
- ✓ 面向"未来程序"的设计
- ✓ 可演进自由度高

局限

- ✗ 与 Claude 平台绑定
- ✗ 需熟悉 Managed Agents API

AWS: OpenAI 兼容优先

优势

- ✓ 生态熟悉度高
- ✓ 迁移路径清晰
- ✓ OpenAI Harness 性能优化

局限

- ✗ 以 OpenAI 模型为核心
- ✗ 跨厂商接口未对齐

安全与身份管理对比

三种思路应对凭据、隔离与审计

维度	Azure	Anthropic	AWS
Agent 独立身份		4/5	
凭据隔离力度	4/5		4/5
网络隔离能力	4/5	4/5	
OBO / 委派授权		3/5	4/5
审计日志完备度	4/5	4/5	

说明:评分仅基于公开文档与设计哲学的相对强度示意,非绝对量化。

可观测性、计费与扩展性对比

运维体感的三大关键差异

可观测性	Azure OpenTelemetry + Foundry Tracing + Evaluators	Anthropic Session 事件日志可完整回溯	AWS AgentCore 评估、监控、Agent 发现
计费模型	Azure 按活动会话的 CPU + Memory 消耗	Anthropic 按 Token + Sandbox 资源	AWS 按 Bedrock 推理 + AgentCore 资源
扩展性	Azure 默认 50 并发会话/订阅,可扩配额	Anthropic Brain 无状态,水平扩展容易	AWS 贴合 AWS 计算与数据规模

平台选型决策框架

基于场景、生态和合规约束的三步决策



PART 04

实战指南

从开发流程到上下文工程,从性能优化到安全实践



Hosted Agent 端到端开发流程

以 Azure Foundry 为例,六阶段开发循环



设计良好 Agent Harness 的 6 大原则

工程经验提炼:稳定接口与可演进实现并重

01 | 稳定接口为先

把接口当成 OS 抽象一样设计,让实现自由演进

02 | 宁可解耦,不要耦合

组件独立失败、独立替换、独立扩展

03 | 不养宠物

把容器、Brain、Hand 都当成可替换的牛

04 | 权限最小化

凭据永不进沙箱,通过代理或 Vault 隔离

05 | 状态外置

事件日志独立存储,可恢复、可回看、可审计

06 | 持续质疑假设

模型升级后,旧的 Harness 兜底逻辑可能变成死代码

上下文工程的关键技巧

在 Session 与 Context Window 之间架桥

常用技巧

Compaction

压缩上下文为摘要,定期回写

Memory Tool

Claude 主动写文件,实现跨会话学习

Context Trimming

选择性删除旧工具结果或思考块

Event Replay

通过 `getEvents()` 回放历史片段

Cache 优化

组织上下文以提升 Prompt Cache 命中

黄金法则

Session $\neq$ Context Window

- ▶ Session 是可恢复的事件存储,Context 是当前推理输入。
- ▶ 把不可逆决策(压缩、裁剪)留给 Harness,让 Session 保持完整。
- ▶ 使用位置切片接口,允许 Brain 回到任意历史点。
- ▶ Harness 在写入 Context 前可任意变换数据。
- ▶ 未来模型可能不需要这些技巧,接口要为变化留余地。

性能优化: TTFT 是关键体感指标

Time-To-First-Token 决定用户的等待感

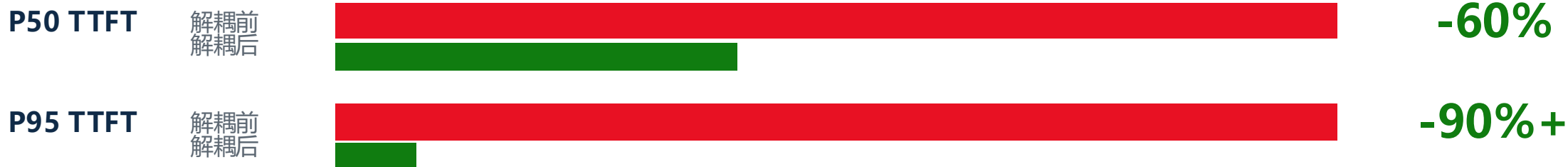
什么是 TTFT?

Time-To-First-Token 测量从会话接受工作到产生第一个响应 Token 的延迟。这是用户最敏锐感知到的时延。

为何会变慢?

将 Brain 放入容器时,所有会话(包括从不调用沙箱的)都要先支付容器启动、Repo Clone、事件拉取的成本。

Anthropic 解耦后的实测改进



常见陷阱与避免方法

从生产实践中提炼的 6 大反模式

! 宠物化容器

失败时手忙脚乱抢救

修复 把容器视为可重启的牛

! 凭据进入沙箱

Prompt Injection 即可窃取 Token

修复 凭据存于 Vault,通过代理调用

! Harness 内嵌资源假设

无法连接客户 VPC

修复 Harness 与 Hand 完全解耦

! 不可逆压缩

压缩后无法恢复关键上下文

修复 保存事件流,延迟决策

! 兼容性死代码

为旧模型加的 Workaround 拖累新模型

修复 定期审视 Harness 假设

! 调试黑盒

WebSocket 流分不清问题来源

修复 组件级追踪 + 隔离故障域

安全最佳实践清单

把 Agent Harness 当作生产应用代码对待

凭据治理

- ✓ 不在镜像中嵌入密钥
- ✓ 使用 Managed Identity
- ✓ 敏感数据使用 Key Vault / Secret Manager

网络隔离

- ✓ 私网部署 Foundry 资源
- ✓ VNet / VPC peering 仅限需要
- ✓ 数据流出边界须审视合规

权限最小化

- ✓ 每 Agent 独立身份
- ✓ RBAC 精细授权
- ✓ OBO 仅用于必要的用户委派

供应链安全

- ✓ 镜像签名与漏洞扫描
- ✓ MCP / 第三方工具白名单
- ✓ 依赖锁定与 SBOM

可追溯性

- ✓ 全量 Action 审计日志
- ✓ 事件流不可篡改
- ✓ 异常告警与回放

Prompt 安全

- ✓ 输入清洗 + Output 审核
- ✓ 敏感工具的人工确认
- ✓ 防 Prompt Injection 模板

未来趋势:Meta-Harness 与可演进架构

Agent 平台的下一个 10 年:接口稳定,实现自由

"Programs as yet unthought of."

操作系统通过将硬件虚拟化为足够通用的抽象,延续了数十年。Managed Agents 走相同道路:对接口有主张,对实现保持中立。

通用 Agent 接口标准化

MCP / A2A / OpenAI Responses 等协议层将持续融合

Agent 即基础设施

Agent 成为云平台一等公民,与 VM、容器、函数并列

脑手分布式部署

Brain 跨云、Hand 在客户内部,实现混合云 Agent

评估与安全闭环

Evaluators + 审计 + 策略统一,生产级可治理

T H A N K Y O U

总结与 Q&A

- Agent Harness 把 LLM 推理之外的工程复杂度收敛到平台层
- Brain / Hands / Session 解耦是 Managed Agents 的核心设计哲学
- Azure / Anthropic / AWS 各擅所长,选型应贴合业务场景
- 稳定的接口比聪明的实现更重要——为未来留下空间